

A. Summarize **one** real-world written business report that can be created from the DVD Dataset from the “Labs on Demand Assessment Environment and DVD Database” attachment.

- My real-world business report will be “**Monthly Revenue By Store**”. This report will show the monthly revenue for each store by using data from the rental, payment, and inventory tables in the DVD database. The detailed section will display each payment record in both stores, and the summary section will consolidate the monthly totals for each store. This report will help monitor store performance and identify seasonal revenue patterns which will in turn help with staffing and budgeting.

A1. Identify the specific fields that will be included in the detailed table and the summary table of the report.

#### **Detailed Table**

The detailed table will contain all fields from the following tables

| Payment Table | Rental Table | Inventory Table |
|---------------|--------------|-----------------|
| payment_id    | inventory_id | store_id        |
| rental_id     |              |                 |
| amount        |              |                 |
| payment_date  |              |                 |

#### **Summary Table**

The summary table will have the following fields

|            |
|------------|
| store_id   |
| revenue    |
| year_month |

A2. Describe the types of data fields used for the report.

| Payment Table   data type | Rental Table   data type | Inventory Table   data type |
|---------------------------|--------------------------|-----------------------------|
| payment_id = integer      | inventory_id = integer   | store_id = integer          |
| rental_id = integer       |                          |                             |
| amount = numeric(5,2)     |                          |                             |
| payment_date = timestamp  |                          |                             |

|                    |
|--------------------|
| store_id = integer |
| revenue = numeric  |
| year_month = date  |

**A3. Identify *at least two* specific tables from the given dataset that will provide the data necessary for the detailed table section and the summary table section of the report.**

The tables used in the business report are payment, rental, and inventory tables.

The payment table contains the payment amounts along with the timestamps of when a movie was rented/paid for.

The rental table is the bridge between payment and inventory. Payment table shares 'rental\_id' with rental table, which will allow us to pull 'inventory\_id' which is also located within the inventory table.

Then 'store\_id' is pulled from the inventory table to assign which store the payment came from.

**A4. Identify *at least one* field in the detailed table section that will require a custom transformation with a user-defined function and explain why it should be transformed (e.g., you might translate a field with a value of N to No and Y to Yes).**

As mentioned above the 'payment\_date' field is stored as a timestamp datatype. This means that the values are stored as the date and the precise time of payment. My business report doesn't need the time of day, therefore, to improve interpretability a user-defined function will extract only the month and year values. We must include both month and year in order to maintain accuracy because we don't want to combine last year's data with the current year and month.

**A5. Explain the different business uses of the detailed table section and the summary table section of the report.**

The detailed table could be used more for the accounting and auditing team. This table could be used to review and verify any transactional history for both stores. This table could be used to handle customer disputes, cash drawer over-shorts, and any unusual activity.

The summary table could be used more for management and forecasting teams. This table is simplified into understandable metrics. This table could be used for budgeting and forecasting monthly revenue, or identifying seasonal trends.

**A6. Explain how frequently your report should be refreshed to remain relevant to stakeholders.**

The report should be refreshed daily. Refreshing the data daily ensures that transaction history and revenue stay accurate and up to date. This allows accounting and managers to track how each store is performing throughout the month. Daily updates allow management to adjust for staffing and make it easier to identify customer disputes, unusual activity, and revenue patterns.

**B. Provide original code for function(s) in text format that perform the transformation(s) you identified in part A4.**

-- Create a UDF that removes the time, stores as a date data type, and stores all days in a month as a group --

```
CREATE OR REPLACE Function delete_time(payment_date TIMESTAMP)
```

```
RETURNS DATE
```

```
LANGUAGE SQL
```

```
AS 'SELECT DATE_TRUNC("month", payment_date)::DATE;;
```

-- Test to see if UDF worked --

```
Select payment_id, payment_date, delete_time(payment_date) AS year_month
```

```
FROM Payment
```

```
Limit 5;
```

-- Since I keep running this over and over, these are checks delete the tables if they already exist --

```
DROP TABLE IF EXISTS detailed_revenue;
```

```
DROP TABLE IF EXISTS summary_revenue;
```

**As you can see above I added a test code to make sure that my UDF worked**

C. Provide original SQL code in a text format that creates the detailed and summary tables to hold your report table sections.

```
-- Part C creates the empty tables --
-- Create empty detailed table --
CREATE TABLE detailed_revenue(
detail_id Serial PRIMARY KEY,
payment_id INTEGER NOT NULL,
rental_id INTEGER NOT NULL,
inventory_id INTEGER NOT NULL,
store_id INTEGER NOT NULL,
payment_date TIMESTAMP NOT NULL,
amount NUMERIC(5,2) NOT NULL);
-- Create empty summary table --
CREATE TABLE summary_revenue(
year_month DATE NOT NULL,
store_id INTEGER NOT NULL,
revenue NUMERIC NOT NULL,
CONSTRAINT summary_revenue_pk
PRIMARY KEY(year_month, store_id));
```

D. Provide an original SQL query in a text format that will extract the raw data needed for the detailed section of your report from the source database.

```
-- Part D Query that will extract the raw data needed for the detailed table --
SELECT
    p.payment_id,
    p.rental_id,
    r.inventory_id,
    i.store_id,
    p.payment_date,
    p.amount
FROM payment p
JOIN rental r
ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
ORDER BY
    p.payment_date, i.store_id
-- This just test to see if everything queries correctly --
LIMIT 10;
```

E. Provide original SQL code in a text format that creates a trigger on the detailed table of the report that will continually update the summary table as data is added to the detailed table.

-- Part E, Trigger function that updates summary after data is added to detailed --

```
CREATE OR REPLACE FUNCTION update_summaryrev()
RETURNS TRIGGER AS $$
DECLARE
v_month_start DATE;
BEGIN
v_month_start := delete_time(NEW.payment_date);
--Updates into summary_revenue, on conflict ensures each row is unique --
INSERT INTO summary_revenue (year_month, store_id, revenue)
Values (v_month_start, NEW.store_id, NEW.amount)
ON CONFLICT (year_month,store_id)
DO UPDATE
SET revenue = summary_revenue.revenue + EXCLUDED.revenue;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger Definition, this tells the function to run after and insert statement --
CREATE TRIGGER trg_update_summaryrev
AFTER INSERT ON detailed_revenue
FOR EACH ROW
EXECUTE FUNCTION update_summaryrev();

-- Testing if trigger worked --
INSERT INTO detailed_revenue
(payment_id, rental_id, inventory_id, store_id, payment_date, amount)
VALUES
(11553,1153,1513,1,'2003-11-05',1.53);
Select * FROM summary_revenue;
```

F. Provide an original stored procedure in a text format that can be used to refresh the data in *both* the detailed table and summary table. The procedure should clear the contents of the detailed table and summary table and perform the raw data extraction from part D.

```
-- Part F storing a procedure that refreshes the data in both the detailed and summary table
--
CREATE OR REPLACE PROCEDURE refresh_revenue()
LANGUAGE plpgsql
AS $$ BEGIN
-- This just deletes anything that was in there before --
TRUNCATE TABLE detailed_revenue;
TRUNCATE TABLE summary_revenue;
-- INSERT FIELDS into detailed revenue --
INSERT INTO detailed_revenue
( payment_id, rental_id, inventory_id, store_id, payment_date, amount )
SELECT p.payment_id, p.rental_id, r.inventory_id, i.store_id, p.payment_date, p.amount
FROM payment p
JOIN rental r
ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
ORDER BY p.payment_date, i.store_id;
END;
$$;
-- Test to see each if detailed and summary tables filled --
-- Detailed Revenue contains each stores daily transaction --
CALL refresh_revenue(); SELECT * FROM detailed_revenue LIMIT 5;
-- Summary table shows each stores monthly revenue --
SELECT * FROM summary_revenue ORDER BY year_month,store_id LIMIT 10;
```

F1. Identify a relevant job scheduling tool that can be used to automate the stored procedure.

A relevant job scheduling tool that can be used to automate the stored procedure would be “pgAgent”. According to the pgAdmin documentation, “pgAgent is a scheduling agent that runs and manages jobs; each job consists of one or more steps and schedules” (pgAdmin, 2024). pgAgent is designed to run in the background and automatically run tasks such as the procedure, refresh\_revenue. For example pgAgent can be configured to run nightly after the stores closes. This ensures that the detailed\_revenue and summary\_revenue tables will be updated in the morning when the store opens.

### **Sources**

pgAdmin. (2024). *Using pgAgent*. Retrieved from  
[https://www.pgadmin.org/docs/pgadmin4/latest/using\\_pgagent.html](https://www.pgadmin.org/docs/pgadmin4/latest/using_pgagent.html)